

Enable authentication globally

If the vast majority of your endpoints should be protected by default, you can register the authentication guard as a **global guard** and instead of using `@UseGuards()` decorator on top of each controller, you could simply flag which routes should be public.

First, register the `AuthGuard` as a global guard using the following construction (in any module, for example, in the `AuthModule`):

```
providers: [  
  {  
    provide: APP_GUARD,  
    useClass: AuthGuard,  
  },  
],
```

With this in place, Nest will automatically bind `AuthGuard` to all endpoints.

```
*****  
  
import { SetMetadata } from '@nestjs/common';  
  
export const IS_PUBLIC_KEY = 'isPublic'  
export const IsPublic = () => SetMetadata(IS_PUBLIC_KEY, true)  
*****
```

Now that we have a custom `@Public()` decorator, we can use it to decorate any method, as follows:

```
@Public()  
@Get()  
findAll() {  
  return [];  
}
```

```
*****
```

And the guard, will be:

```
import { CanActivate, ExecutionContext, Injectable, UnauthorizedException } from
 '@nestjs/common';
import { JwtService } from '@nestjs/jwt';
import { Request } from 'express';
import { jwtConstants } from './constants';
import { Reflector } from '@nestjs/core';
import { IS_PUBLIC_KEY } from 'src/public/public.decorator';
@Injectable()
export class AuthGuard implements CanActivate {

  constructor(private readonly jwtService: JwtService, private readonly
reflector: Reflector){}

  async canActivate(
    context: ExecutionContext,
  ): Promise<boolean> {

    const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      context.getHandler(),
      context.getClass()
    ])

    if(isPublic) {
      return true
    }

    const request = context.switchToHttp().getRequest()
    const token = this.extractTokenFromHeaders(request);
    if(!token) {
      throw new UnauthorizedException()
    }

    try {
      const payload = await this.jwtService.verifyAsync(token, {
        secret: jwtConstants.secret,
      })
      request["user"] = payload
    } catch {
      throw new UnauthorizedException()
    }

    return true;
  }
}
```

```

extractTokenFromHeaders(request: Request): string|undefined {
  const [type, token] = request.headers["authorization"]?.split(' ') ?? []
  return type == 'Bearer' ? token : undefined
}
}
}

*****

```

To use passport with nestjs:

Run command: `npm install --save @nestjs/passport passport passport-local`

Run command: `npm install --save-dev @types/passport-local`

In vanilla Passport, you configure a strategy by providing two things:

1. A set of options that are specific to that strategy. For example, in a JWT strategy, you might provide a secret to sign tokens.
2. A "verify callback", which is where you tell Passport how to interact with your user store (where you manage user accounts). Here, you verify whether a user exists (and/or create a new user), and whether their credentials are valid. The Passport library expects this callback to return a full user if the validation succeeds, or a null if it fails (failure is defined as either the user is not found, or, in the case of passport-local, the password does not match).

With `@nestjs/passport`, you configure a Passport strategy by extending the `PassportStrategy` class. You pass the strategy options (item 1 above) by calling the `super()` method in your subclass, optionally passing in an options object. You provide the verify callback (item 2 above) by implementing a `validate()` method in your subclass.

HINT

We can pass an options object in the call to `super()` to customize the behavior of the passport strategy. In this example, the passport-local strategy by default expects properties called `username` and `password` in the request body. Pass an options object to specify different property names, for example: `super({ usernameField: 'email' })`. See the [Passport documentation](#) for more information.

Our **Passport local strategy** has a default name of **'local'**. We reference that name in the **@UseGuards()** decorator to associate it with code supplied by the **passport-local package**. This is used to disambiguate which strategy to invoke in case we have multiple Passport strategies in our app (each of which may provision a strategy-specific **AuthGuard**).

Passport automatically creates a **user** object, based on the value we return from the **validate()** method, and assigns it to the **Request** object as **req.user**.

To use logout method of passport, we need to install: **npm install express-session**

Then: **npm install -D @types/express-session**

To use passport with jwt we need to install: **npm i --save passport-jwt**

The auth module for passport jwt is:

```
import { Module } from '@nestjs/common';
import { PassportModule } from '@nestjs/passport';
import { JwtModule } from '@nestjs/jwt';
import { AuthController } from './auth.controller';
import { AuthService } from './auth.service';
import { UsersModule } from 'src/users/users.module';
import { jwtConstants } from './constants';
import { LocalStrategy } from './local.strategy';
import { JwtStrategy } from './jwt.strategy';

@Module({
  controllers: [AuthController],
  providers: [AuthService, LocalStrategy, JwtStrategy],
  imports: [UsersModule, JwtModule.register({
    global: true,
    secret: jwtConstants.secret,
    signOptions: { expiresIn: '3600s' }
  }), PassportModule],
  exports: [AuthService],
})
```

```

}))
export class AuthModule {}

```

The local strategy is:

```

import { Strategy } from 'passport-local'
import { PassportStrategy } from '@nestjs/passport'
import { AuthService } from './auth.service';
import { Injectable, UnauthorizedException } from '@nestjs/common';

@Injectable()
export class LocalStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: AuthService) {
    super();
  }

  async validate(username: string, password: string): Promise<any> {
    const user = await this.authService.validateUser(username, password)
    if(!user) {
      throw new UnauthorizedException()
    }
    return user;
  }
}

```

The jwt strategy is:

```

import { Injectable } from "@nestjs/common";
import { PassportStrategy } from "@nestjs/passport";
import { Strategy, ExtractJwt } from "passport-jwt";
import { jwtConstants } from "./constants";

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: jwtConstants.secret,
    });
  }
}

```

```

    async validate(payload: any) {
      return {
        userId: payload.sub, username: payload.username
      }
    }
  }
}

```

The auth service is:

```

import { Injectable, UnauthorizedException } from '@nestjs/common';
import { UsersService } from 'src/users/users.service';
import { JwtService } from '@nestjs/jwt'

@Injectable()
export class AuthService {
  constructor(private readonly usersService: UsersService, private readonly
jwtService: JwtService) {}

  async signIn(username: string, pass: string): Promise<{ access_token: string
}> {
    const user = await this.usersService.findOne(username)
    if(user.password !== pass) {
      throw new UnauthorizedException();
    }

    const payload = { sub: user.userId, username: user.username }
    return {
      access_token: await this.jwtService.signAsync(payload),
    }
  }

  async validateUser(username: string, pass: string): Promise<any> {
    const user = await this.usersService.findOne(username)
    if(user && user.password == pass) {
      const { password, ...result } = user
      return result
    }
    return null
  }

  async login(user: any): Promise<{access_token: string}> {
    const payload = { username: user.username, sub: user.userId }
    return {
      access_token: await this.jwtService.signAsync(payload)
    }
  }
}

```

```

    }
  }
}

```

The auth controller:

```

import { Body, Controller, Get, HttpStatusCode, HttpStatus, Post, Redirect, Request,
UseGuards } from '@nestjs/common';
import { AuthService } from './auth.service';
import { SignInDto } from './dto/signin.dto';
import { AuthGuard } from './auth.guard';
import { LocalAuthGuard } from 'src/local-auth/local-auth.guard';
import { JwtAuthGuard } from 'src/jwt-auth/jwt-auth.guard';

@Controller('auth')
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @HttpCode(HttpStatus.OK)
  @Post('login')
  signIn(@Body() signInDto: SignInDto) {
    return this.authService.signIn(signInDto.username, signInDto.password);
  }

  @HttpCode(HttpStatus.OK)
  @UseGuards(LocalAuthGuard)
  @Post('passport/login')
  async login(@Request() req) {
    return this.authService.login(req.user)
  }

  @HttpCode(HttpStatus.OK)
  @UseGuards(LocalAuthGuard)
  @Post('passport/logout')
  async logout(@Request() req) {
    return req.logout(function (err) {
      if(err){
        throw err
      }
      return Redirect('/')
    })
  }
}

```

```

@UseGuards(AuthGuard)
@Get("profile")
async profile(@Request() req) {
    return req.user
}

@UseGuards(JwtAuthGuard)
@Get("passport/profile")
async passportProfile(@Request() req) {
    return req.user
}
}

```

The graphql guard is:

```

import { ExecutionContext, Injectable } from '@nestjs/common';
import { GqlExecutionContext } from '@nestjs/graphql';
import { AuthGuard } from '@nestjs/passport';
import { Observable } from 'rxjs';

@Injectable()
export class GqlAuthGuard extends AuthGuard('jwt') {
    canActivate(
        context: ExecutionContext,
    ): boolean | Promise<boolean> | Observable<boolean> {
        return super.canActivate(context)
    }

    getRequest(context: ExecutionContext) {
        const ctx = GqlExecutionContext.create(context)
        return ctx.getContext().req
    }
}

```

The jwt auth guard is:

```
import { ExecutionContext, Injectable, UnauthorizedException } from
 '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';
import { Observable } from 'rxjs';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  canActivate(context: ExecutionContext): boolean | Promise<boolean> |
  Observable<boolean> {
    return super.canActivate(context)
  }

  handleRequest(err: any, user: any, info: any, context: ExecutionContext,
  status?: any) {
    if(err || !user) {
      throw new UnauthorizedException()
    }
    return user
  }
}
```

The local auth guard is:

```
import { Injectable } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

@Injectable()
export class LocalAuthGuard extends AuthGuard('local') {
}
```

The current user decorator for graphql (search for example about it using deepseek, qwen, ...etc):

```
export const CurrentUser = createParamDecorator(
  (data: unknown, context: ExecutionContext) => {
    const ctx = GqlExecutionContext.create(context)
    return ctx.getContext().req.user
  },
);
```